

HeteroSFL: Split Federated Learning With Heterogeneous Clients and Non-IID Data

Xing Chen¹, Jingtao Li¹, *Member, IEEE*, Deliang Fan¹, *Senior Member, IEEE*,
and Chaitali Chakrabarti², *Fellow, IEEE*

Abstract—Split Federated Learning (SFL) is an emerging privacy-preserving decentralized learning scheme which splits a machine learning model between client and server such that most of the computations are offloaded to the server. While SFL has low computation cost on the client side, it has high communication cost. Existing SFL schemes focus on reducing the communication cost for homogeneous clients. However, a more realistic scenario is when clients are heterogeneous and process data with different distributions. In this article, we focus on client-level heterogeneity caused by different communication data rates. We propose HeteroSFL, the first SFL framework with heterogeneous clients that handles non-IID data with label distribution skew across groups of clients. HeteroSFL compresses data with different compression factors in low-end and high-end groups using narrow and wide bottleneck layers (BL), respectively. It provides a mechanism to address the challenge of aggregating different-sized BL models and utilizes bidirectional knowledge sharing (BDKS) to address the overfitting issue caused by different label distributions across high- and low-end groups of clients. Our experimental results show that HeteroSFL achieves significant training time reduction with minimum accuracy loss compared to competing methods. Specifically, it can reduce the training time of SFL by 16x to 256x with 1.24% to 5.59% accuracy loss for VGG11 on CIFAR10 for non-IID data.

Index Terms—Heterogeneous clients, label distribution skew, non-IID data, split federated learning (SFL).

I. INTRODUCTION

DEEP neural network (DNN)s are being extensively used in a wide variety of applications including facial recognition, healthcare, and traffic flow optimization, etc. [1], [2], [3]. Data privacy has become a big concern in centralized training of such models, since raw data usually contains sensitive information which has to be sent to the central server. Recently, several decentralized-training schemes, including federated learning (FL) [4], split learning (SL) [5], and its parallel version—split FL (SFL) [6], have been proposed to train DNN models without sharing raw data with the server. In FL, the edge clients train the whole model with their

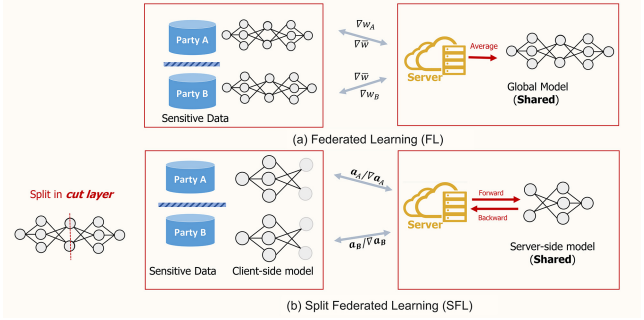


Fig. 1. Training process of (a) FL and (b) SFL. In FL, clients send the gradients ∇w_A and ∇w_B to the server, and the server sends the averaged gradients $\bar{w}$ back to clients. In SFL, clients send the intermediate activations a_A and a_B to the server, and the server sends the gradients of the intermediate activations ∇a_A and ∇a_B back to the clients.

private data and send the model parameters to the server side; the server then computes the global model through aggregation. Fig. 1(a) describes the training process in FL. In both SL and SFL, the model is partitioned into a small client-side model and a big server-side model. The clients only compute the forward and backward propagation of the small client-side model and the server handles the forward and backward computation of the server-side model. In SL, the clients are trained in a round-robin fashion, where the latest client-side model parameters are sent to the next client after processing every batch of samples [5]. This sequential process significantly slows down the training process.

SFL [6] is a parallel version of SL, where all client-side models are trained in parallel, and aggregated at the end of each training epoch, as described in Fig. 1(b). Both SL and SFL offload most of the computations to the server and are thus more suitable than FL for resource-constrained clients. While SFL has significantly lower training time compared to SL, both SL and SFL suffer from high client-server communication overhead. This is because the activations and gradients of every batch of samples have to be communicated between the client and the server. Thus, to reduce the communication cost, the volume of activations and gradients have to be significantly reduced. Unfortunately, existing data compression methods applied to SFL like quantization [7], Top-k sparsity [8], and selective updating [9] result in a significant accuracy drop. For example, the quantization method in [7] reduces transmitted/received data size by 2x at the cost of 5% accuracy loss. Recently, an end-to-end trained bottleneck layer (BL) method was proposed in [10] and [11] that adds a

Received 20 November 2024; revised 5 March 2025 and 16 April 2025; accepted 13 May 2025. Date of publication 21 May 2025; date of current version 25 July 2025. This work was supported in part by the Army Research Laboratory under Cooperative Agreement under Grant W911NF-23-2-0225. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Laboratory or the U.S. Government. (Corresponding author: Chaitali Chakrabarti.)

The authors are with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ 85281 USA (e-mail: xchen382@asu.edu; jingtao1@asu.edu; dfan@asu.edu; chaitali@asu.edu).

Digital Object Identifier 10.1109/IJOT.2025.3572393

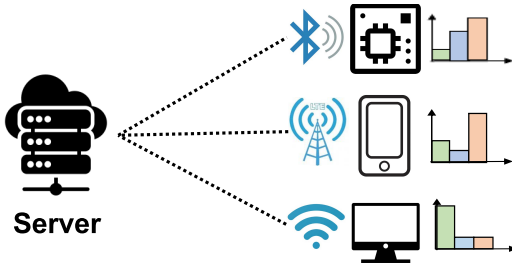


Fig. 2. SFL with heterogeneous clients supporting different data rates and different data distributions. The colored bars represent the number of samples in each class.

convolution layer with fewer output channels (encoder) on the client side and the corresponding mirrored structure (decoder) on the server side. The BL method has been shown to reduce the data size while maintaining good accuracy performance for both inference and training. Therefore, we utilize BL as the main data compression method in this article.

Existing works in SFL consider only homogeneous clients. In a realistic scenario, the clients are usually heterogeneous, where different clients have different data rates and operate on data with different label distributions, as shown in Fig. 2. For example, in recommendation systems, the client devices are heterogeneous and vary significantly in their system capabilities; the data on each client is also different from each other [12]. Similar situations occur in smart cities [13], intelligent transportation infrastructure [14] and smart healthcare systems [15], where client-side devices operate on very different data under diverse network environments. Clients can also be heterogeneous in terms of their computing power. We do not consider that aspect since in SFL the computation cost on the client side is usually a lot smaller than the communication cost [7], [16]. The computation cost becomes comparable with communication cost when the data rates increase to tens of Mbs [17]. In such cases, the existing algorithms proposed for addressing the computation heterogeneity in FL [18], [19], [20], [21], [22] can be used to address the computation heterogeneity in SFL.

In this article, we cluster the clients that support similar data rates into one group, so a high-end group of clients communicates with high data rates while a low-end group communicates with low data rates. We consider the following two scenarios.

- 1) *Dynamic Group Affiliation*: All clients use the same communication protocol. Assuming that the wireless channel is slow fading and exhibits wide-sense stationary uncorrelated scattering (WSSUS) properties, if the Doppler Shift is minimal, the average data rate for each client does not change over an epoch. In such a scenario, a client can move in and out of high-end/low-end groups at epoch-level granularity.
- 2) *Static Group Affiliation*: Each client supports a specific communication protocol. For example, some clients only support low communication rate protocols, such as BLE and Zigbee, and others support communication protocols with high data rates, such as WiFi and LTE. Here, the clients associated with any particular group are fixed during the whole training process.

In terms of data distribution, we mainly consider the label distribution skew.

- 1) *IID Data*: The number of samples per class is the same for all clients;
- 2) *Non-IID Data*: The number of samples per class differs from client to client. Due to groups of clients subscribing to different data rates, the non-IID data can lead to the following two types of label skewness.
 - a) *Interclient Data Distribution Skew*: The number of samples per class differs from client to client [23], a problem that has been well studied in FL [24], [25], [26], and can be extended to alleviate the interclient label skew in SFL to some extent.
 - b) *Intergroup Data Distribution Skew*: The number of samples per class differs across groups of clients, a phenomenon that has been observed in FL recommendation systems [12]. This has not been considered in prior works on heterogeneous FL systems [18], [19], [27]. Note that the intergroup skew does not occur in the dynamic group affiliation case since the clients' affiliation changes over time.

In SFL, the server cannot start processing until it receives activations from all participating clients. Therefore, the server has to wait for data from the group with the slowest data rate, which increases the overall training time significantly. Since the communication time of SFL is dominated by the low-end group of clients, reducing the data size transmitted/received by low-end group can help reduce the training time. Unfortunately, existing BL methods [10], [11] always implement identical-sized BLs (IdBLs) for all clients. Since the communication data of both high-end and low-end groups is compressed by the same amount, existing BL works fail to address the long training time problem caused by the low-end groups of clients. To reduce the training time, we propose that the low-end group of clients with slow data rates compress the data by a large ratio using narrow BLs (designed using fewer channels in the convolution layer). In contrast, the high-end group of clients with fast data rates can use wider BLs (designed using more channels) to maintain high accuracy performance.

However, the use of nonidentical BL structures across different groups of clients causes two challenges. First, the parameters of the different-sized BL models cannot be directly aggregated using averaging due to the differences in the BL structures. The second challenge is the label distribution skew caused by non-IID data across clients and groups of clients. When different groups use nonidentical BL, the intergroup label skew makes the model with nonidentical BL overfit to the data within their respective groups, leading to significantly lower accuracy.

In this article, we first design Heterogeneous BL (HetBL) to reduce the training time by compressing the transmitted data in different groups by different amounts through use of different-sized BLs. The challenge of aggregating differently structured BLs is addressed by setting the narrow BLs to be subnetworks of wide BLs. While such a method is sufficient

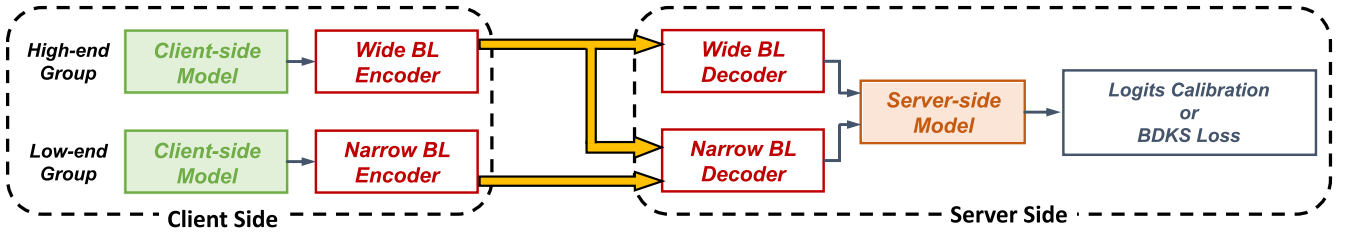


Fig. 3. Proposed HeteroSFL method, where the data of different clients are compressed by different ratios through use of different-sized BLs (HetBL). To mitigate the over-fitting due to non-IID data, the loss function is replaced with LC ($\mathcal{L}_k^{\text{cal}}$) or BDKS ($\mathcal{L}_k^{\text{BDKS}}$). Only the forward path is shown here.

for improving accuracy with low training time for the dynamic group affiliation case, it alone is not sufficient for the static group affiliation case. The static affiliation case suffers from the intergroup label skew problem which can be addressed by the proposed bidirectional knowledge sharing (BDKS) method. In BDKS, the wide BL model learns knowledge on underrepresented classes from the narrow BL model which is trained by the entire dataset, and the narrow BL model learns from the wide BL model to avoid gradient interference between wide and narrow BL models. For both cases, we utilize the existing LC method [26] to mitigate the interclient label skew. Fig. 3 describes the proposed HeteroSFL method.

Our extensive experiments show that for the static group affiliation case, HeteroSFL can reduce training time of SFL by $16\times$ to $256\times$ with only 1.24% to 5.59% accuracy loss for VGG11 on CIFAR10 for non-IID data. For the same training time reduction, HeteroSFL achieves accuracy improvement over other schemes; the improvement is 30.07% over SampleReduce (SR) and 7.16% over Top-k sparsity [8]. For the dynamic group affiliation case, the proposed HetBL and HeteroSFL have similar performance. HetBL outperforms SR, IdBL [11], and Top-K sparsity by 54.26%, 9.51%, and 13.24%, respectively, with $256\times$ communication reduction for VGG11 on CIFAR-10.

Our contributions are as follows.

- 1) HetBL is proposed to reduce the training time of heterogeneous clients by using different-sized BLs.
- 2) BDKS is proposed to address the low accuracy caused by intergroup label distribution skew when using non-identical sized BL for static groups of clients.
- 3) HeteroSFL, a combination of HetBL and BDKS, is designed to reduce the training time with minimum accuracy loss in heterogeneous clients. For static group affiliation, both components are necessary, while only HetBL is required for dynamic group affiliation.
- 4) Extensive experiments for IID and non-IID data distributions for both static and dynamic group affiliation cases demonstrate the advantages of HeteroSFL compared with other state-of-the-art methods.

II. RELATED WORK

SFL [6] is a parallel version of SL that was first proposed in [5]. It splits the neural network model into a small client-side model and a big server-side model, and thereby offloads most of the computations to the server, as shown in Fig. 1(a). Similar to FL, SFL consists of two steps, such as local training

and aggregation. In local training, the client-side model of client k is trained by its local data $\mathbf{D}_k$ as follows:

$$\min_{f, g_k} \sum_{\{x, y\} \in \mathbf{D}_k} \mathcal{L}(f(g_k(x)), y) \quad (1)$$

where $\mathcal{L}(\cdot, \cdot)$ is the loss function and $g_k(\cdot)$ and $f(\cdot)$ represents the client-side model of the k th client and the server-side model. For convenience, we use $f_k(\cdot)$ to represent $f(g_k(\cdot))$ in the remainder of this article. At the end of every epoch of local training, the client-side models from all clients are aggregated on the server side by averaging and the global client-side model is broadcast back to the clients to initialize the client-side model for the next epoch.

Data Compression Methods in SFL: In SFL, the transmission of activations and gradients for every sample in every epoch results in a high communication overhead. Recent developments in SFL include reducing the communication overhead by totally eliminating the gradient transmission [32], [33], and reducing the number of training epochs [9], [11], which cause significant accuracy loss. Existing works to compress the activations/gradients of DNN using JPEG [10], quantization [7], [9], and sparsity [8] all suffer from high accuracy loss. End-to-end trainable BL, proposed in [10] and [11] to compress the data size using trainable convolution layers, achieve good accuracy with large data compression. We build upon this promising technique to reduce the transmitted/received data size.

FL/SFL With Heterogeneous Clients: In FL, computation overhead is a major concern and the low-end group usually takes longer, increasing the overall training time. Some works reduce the computation time of low-end group in FL by reducing the number of local training epochs [34], [35] and reducing the number of processed samples [36], [37]. The same reduction methods can be extended to SFL to reduce communication time but suffer from accuracy loss, as shown in Section V. Another promising method that is used in FL is asynchronous training, which allows the server to aggregate models asynchronously [38], [39]. While these methods reduce the training time, they require additional management to address biased convergence of the global model.

To reduce the computation time of low-end client group in FL, recent papers in FL propose that the low-end group train a small network compared to the high-end group [18], [19], [20], [21], [22], [40], [41], [42], [43]. In many cases, this smaller network is a subnetwork of the full global model [18], [19], [20], [41], [42], [43] with its structure being either

TABLE I
SUMMARY OF THE PRIMARY NOTATIONS USED IN THIS ARTICLE

Notation	Definition
$\mathbf{K}, \mathbf{K}_{low}, \mathbf{K}_{high}$	the set of all, low-end and high-end clients, respectively
k	client k
$\mathbf{D}_k$	dataset of client k
(x, y)	x is the sample data and y is the corresponding label
e_k^p	encoder of p -channel BL in client k
e^p	global encoder of p -channel BL after aggregation
$e^p[p_N : p_W]$	p_N -th to p_W -th channels in p -channel BL
d^p	decoder of p -channel BL on the server side
γ_k	aggregation coefficient of client k which is proportional to the number of samples in client k
$f_k^p(x)$	logits after the process of the k -th client-side model and the server-side model using p -channel BL
$f_{k,i}^p(x)$	logit of class i after the process of the k -th client-side model and the server-side model using p -channel BL
$ f(x) $	size of $f(x)$
$\mathcal{R}$	set of underrepresented classes processed by high-end group
$\mathcal{J}$	set of well-represented classes processed by high-end group
$\mathcal{L}_k^{N2W}, \mathcal{L}_k^{W2N}$	Narrow-to-Wide and Wide-to-Narrow loss function for client k
α	BDKS hyper-parameter which controls the magnitude of N2W knowledge sharing
θ	BDKS hyper-parameter which controls the percentage threshold of underrepresented class
τ	logit calibration hyper-parameter which controls the magnitude of label margin

TABLE II
SUMMARY OF REPRESENTATIVE STATE-OF-THE-ART WORKS ON SYSTEM AND DATA HETEROGENEITY IN SFL

Reference	System Hetero.		Data Hetero.		Comments
	Comp. Hetero.	Comm. Hetero.	Inter-client	Inter-group	
SFL [6]	×	×	×	×	Baseline
MocoSFL [28]	×	×	✓	×	Increasing sync. frequency
AdaSFL [29]	✓	✓	×	×	Adjusting batch sizes; Local updating frequency
ParallelSFL [30]	✓	✓	✓	×	Clustering similar clients; Adjusting local updating frequency
PipeSFL [31]	✓	✓	×	×	Prioritizing slow clients; Async. training within rounds
HeteroSFL (Ours)	×	✓	✓	✓	HetBL for data compression; BDKS for handling non-IID data

static, as in HeteroFL and Fjord [18], [19], or dynamic, as in Fedrolex [43]. If the smaller network models differ significantly from the full model, the smaller models operate on public datasets on the server side and utilize knowledge distillation to help generate the global models [21], [22], [40]. Since the client-side computation in SFL is quite small, we do not consider these. Instead, we focus on reducing the communication cost by compressing the transmitted/received data using BL.

Recently, several schemes have been proposed to tackle heterogeneity in SFL systems. ParallelSFL [30] tackles system heterogeneity by grouping clients with similar system capabilities and adjusting the local updating frequency of the different client groups. PipeSFL [31] addresses the system heterogeneity by prioritizing slow clients and enabling asynchronous training within rounds. AdaSFL [29] assigns smaller batch sizes to low-end clients to balance the training time across clients. All three methods are orthogonal to HeteroSFL which approaches the problem from an algorithmic perspective. A combination of these methods can thus be used to enhance the performance of SFL schemes that handle system-level heterogeneity. We summarize the key features of the related SFL works in Table II.

Label Distribution Skew: The data processed by different clients usually suffers from interclient label distribution skew, where the number of samples per class differs from client to client [44], hampering convergence as presented in [24], [25], and [44]. In SFL and FL, the interclient label skew causes the client-side model to overfit the local data. In SFL, this is addressed by increasing the frequency of synchronization at the cost of more communication overhead in [28] and training the client-side model using a more evenly distributed subset of the original dataset in [45]. In FL, some approaches address the label skew problem by restricting local updates to the global model [24], [34], [35], [46], [47], [48]; others attempt to rectify the data distribution among clients by pruning datasets through machine learning [49], using out-of-distribution datasets [25], [50] and synthesized data [51], [52], [53] or using generative model to derive partitioned performance-sensitive features [54]. However, all these methods incur significant computation and communication overhead on the resource-limited clients and are unacceptable for SFL. A recently proposed method [26] adjusts logit values of minority classes to address the non-IID issue without additional overhead. Thus, we extend the use of LC [26] to SFL to mitigate the effect of interclient label skew in HeteroSFL. Note that none of the above methods consider the problem of intergroup label skew.

III. BACKGROUND AND PROBLEM DESCRIPTION

In this article, we focus on training-time reduction with minimum accuracy loss in an SFL system with heterogeneous clients processing data with interclient and intergroup label skew. Table I summarizes the notations used in this article. The overall training time of SFL is calculated by

$$T = \max_{k \in \mathbf{K}} ((\text{comm}_k + \text{comp}_k)) \times n_{\text{batches}} \times n_{\text{epochs}} \quad (2)$$

$$\text{comm}_k = \frac{2 \times |g_k(x)| \times \text{batch_size}}{r_k} \quad (3)$$

comm_k and comp_k are the communication and computation time of client k to process a batch of samples. comm_k is a function of the data size transmitted/received per sample, batch_size is the number of samples in a batch and r_k is the data rate of client k . The size of data that is transmitted/received per sample includes the client-to-server activations and server-to-client gradients, and is given by $2 \times |g_k(x)|$.

A. Training Time

In standard SFL, the computation time is significantly smaller compared to the communication time. For example, in an SFL system for training VGG11 on CIFAR10, when the client-side model includes the first 3 convolution layers, the computation time is 0.69 s when the client is a Raspberry Pi. The communication data for a batch of 13 samples is around 13Mb, which takes around 1.63 s using WiFi with 8 Mb/s and increases to 6.5 s using WiFi with 2Mb/s. The training time is determined by the maximum time to process one batch of samples across clients since the server has to wait for activations from all clients. Therefore, the overall training time is determined by the communication time of the low-end group in heterogeneous SFL.

B. Dynamic and Static Group Affiliation

In dynamic group affiliation, the clients use the same communication protocol, but the transmission channel condition varies slowly with time. We assume that the wireless channel is slow fading and exhibits WSSUS properties at the epoch level. Assuming that the coherence time exceeds the duration of an epoch, then similar to FL studies [9], [55], we assume that the average data rate for each client does not change over an epoch. To simplify the problem, we also assume that the data rates for both uplink and downlink are the same.

In static group affiliation, the heterogeneous clients subscribe to different communication protocols with different data rates based on their resource constraints. Thus, the data rates of clients within each group are consistent throughout the whole training process. Furthermore, the clients in a group are fixed, introducing a new challenge stemming from the skewed label distribution.

C. Bottleneck Layers

Compressing activations/gradients ($|g_k(x)|$ in (3) of low-end group reduces the communication time, and thereby reducing the overall training time. The BL technique [10], [11] introduces two convolution layers, one at the end of client-side model to compress the channel size of activations from $\bar{p}$ to p , and the other one at the beginning of server-side model to decompress the activation from p back to $\bar{p}$. Note that the number of channels of gradients is also compressed since it has the same size as that of activations. The BL is trained together with the whole network in an end-to-end manner and the extended network is used during inference. We denote the BL-generating activations of p channels as p -channel BL. Larger p represents smaller compression ratio and thus higher

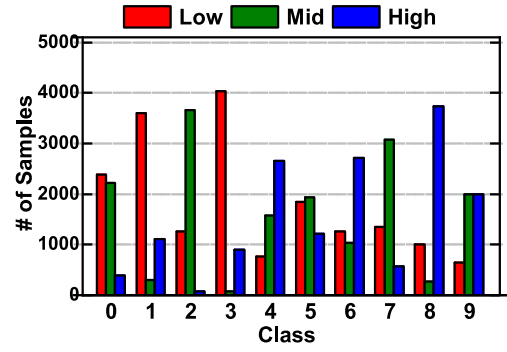


Fig. 4. Example of intergroup label skew caused by system-induced data heterogeneity. There are three groups of clients: low-end, middle-end, and high-end. The number of classes is 10.

accuracy performance since more information is sent to the server.

D. Interclient and Intergroup Label Distribution Skew

Interclient label distribution skew occurs when the number of samples per class differs from client to client [23]. The interclient label skew has been well studied in FL [24], [25], [26], and can be extended to alleviate the interclient label skew in SFL to some extent.

However, the existing works do not consider the intergroup label distribution skew, which is a specific problem that only happens when different groups of clients train models with different structures. Fig. 4 shows an example of the intergroup label skew with 10 classes, where the label distribution across high-, mid- and low-end groups is quite different due to system-induced data heterogeneity. As a result, models trained by different groups of clients tend to overfit the label distribution of its own group, as will be shown in Section IV-B. A recent study in FL recommendation system [12] demonstrated that when groups with different system capabilities train models with different structures, the intergroup label skew makes the model over-fit to the data within their respective groups, significantly compromising the accuracy of existing heterogeneous FL systems [18], [19], [27]. The existing methods for interclient label distribution [23], [25], [46], [52] cannot address the intergroup label distribution problem, as will be described in Section IV-B.

IV. HETEROSFL

In this section, we describe the proposed HeteroSFL framework in detail. For ease of understanding, the framework is described for 2 groups of clients first and then extended to more groups. HeteroSFL achieves high accuracy with low training time for training with data that has interclient and intergroup label skew. It has two components: 1) HetBL (Section IV-A) to address the different compression ratios of different groups through use of different-sized BL and 2) BDKS (Section IV-B) to alleviate the overfitting caused by intergroup label skew. While intergroup label skew is a problem in static group affiliation case, it is not so in the dynamic group affiliation case. So to achieve high accuracy

with significant training time reduction, for the dynamic group affiliation case, HetBL is sufficient but for the static group affiliation case, both HetBL and BDKS are necessary.

A. HetBL

Since the amount of data to be compressed by different groups of clients (the high-end and low-end groups) is different, we propose to use different-sized BLs for different groups. For a two-group system, we use wide BL for high-end group and narrow BL for low-end group. Once the network is trained, the wide BL model is used for inference by all clients to achieve higher accuracy. Since the size of data transmitted during inference is much smaller than in training, this is an acceptable compromise.

We define $e_k^p \in \mathbb{R}^{\tilde{p} \times p \times w \times w}$ as the encoder convolution layer of the p -channel BL in the client-end and $d^p \in \mathbb{R}^{p \times \tilde{p} \times w \times w}$ as the decoder convolution layer in the server-end. The wide BLs are trained only by the high-end group, and narrow BLs are trained only by the low-end group. Due to the different structures, the parameters of wide and narrow BLs cannot be easily aggregated.

In order to be able to aggregate parameters of wide and narrow BLs, we derive the narrow p_N -channel BL as a subnetwork of wide p_W -channel BL. Fig. 5 shows the aggregation of HetBL where the first p_N channels of wide BLs in the high-end group are aggregated with the narrow BLs in the low-end group, and the remaining $(p_W - p_N)$ channels of wide BLs (in the high-end group) are separately aggregated. Thus

$$e^{p_W}[1 : p_N] = e^{p_N} = \sum_{k \in K_{\text{low}}} \gamma_k \cdot e_k^{p_N} + \sum_{k \in K_{\text{high}}} \gamma_k \cdot e_k^{p_W}[1 : p_N] \quad (4)$$

$$e^{p_W}[p_N : p_W] = \sum_{k \in K_{\text{high}}} \gamma_k \cdot e_k^{p_W}[p_N : p_W] \quad (5)$$

where e^{p_N} and e^{p_W} are the global p_N -channel and p_W -channel BL after aggregation, respectively. K_{low} and K_{high} correspond to the set of clients in the low-end and high-end group, respectively, and γ_k is proportional to the number of samples in client k .

When the low-end and high-end group use different-sized BLs as in HetBL, we expect its accuracy to be better compared to IdBL, since the high-end group does not compress the data as much. Fig. 6(a) shows the accuracy of IdBL [11], DiffBL and HetBL in dynamic group affiliation case for both IID and non-IID data distributions. We include DiffBL, a naive different-sized BL method where the narrow/wide BL models solely aggregate with other narrow/wide BL models but not with wide/narrow BL models, to demonstrate the importance of aggregating both narrow and wide BL models together. HetBL achieves the highest accuracy for both IID and non-IID cases as expected, highlighting the importance of using different-sized BL and the ability to aggregate narrow and wide BL models.

B. Bidirectional Knowledge Sharing

Fig. 6(b) shows the accuracy of IdBL [11], DiffBL, and HetBL in the static group affiliation case for both IID and non-IID data distributions. HetBL achieves 8.58% and 39.53% higher accuracy compared with IdBL and DiffBL for the IID case. However, the accuracy of HetBL is 7.49% lower than IdBL when the data distribution is non-IID. In static group affiliation, for the non-IID case, both interclient and intergroup label skew is present. To mitigate the interclient label skewness, we add LC [26] on top of HetBL. While HetBL+LC improves the accuracy of non-IID case by 5.09%, it is still worse than IdBL+LC.

To better understand the cause for poor performance, Fig. 6(c) shows the class-wise accuracy comparison of IdBL+LC and HetBL+LC, and the number of samples per class processed by the high-end group. While the accuracy of classes 0 to 3 using HetBL is higher than IdBL, the accuracy drops in classes 4 to 9 that have insufficient number of samples. Therefore, the wide BL model over-fits the underrepresented classes during SFL training. To better understand the cause for over-fitting, consider equations (4) and (5) where only the first several channels of wide BL are trained by both groups, and the remaining channels of wide BL are trained only by the high-end group. With intergroup label skew, the number of samples of some classes is insufficient as shown in Fig. 6(a). Training wide BL model with data from underrepresented classes results in lower accuracy, as shown in Fig. 6(c).

We define $\mathcal{R}$ to be the set of underrepresented classes processed by high-end group where the number of samples per class is less than $\theta\%$ of its share in the global dataset, and $\mathcal{J}$ the remaining set of classes with sufficient number of samples. Since the high-end group does not have enough samples from the underrepresented $\mathcal{R}$ classes to train its wide BL model, the wide BL should learn about $\mathcal{R}$ classes from other models that have been trained using sufficient number of samples from $\mathcal{R}$. Note that the narrow BL model is a subnetwork of the wide BL, and thus can be trained by the data both from low-end and high-end groups.

We propose BDKS where the narrow BL model teaches the wide BL model (Narrow-to-Wide) and the wide BL model teaches the narrow BL model (Wide-to-Narrow), as shown in Fig. 7. The narrow BL model contains more knowledge about underrepresented $\mathcal{R}$ classes since it is trained by sufficient number of samples, thus in the Narrow-to-Wide direction, the narrow BL model shares the knowledge of $\mathcal{R}$ classes with the wide BL model to avoid wide BL model over-fitting to the underrepresented $\mathcal{R}$ classes. In contrast, the wide BL model contains more knowledge about $\mathcal{J}$ classes due to it having more number of channels in its structure; therefore, in the wide-to-narrow direction, the wide BL model shares the knowledge of $\mathcal{J}$ classes with the narrow BL model.

Narrow-to-Wide (N2W) Knowledge Sharing: The wide BL model learns from the narrow BL model using logits of $\mathcal{R}$ classes generated by narrow BL model. Let $f_k^{p_N}(x) = \{f_{k,1}^{p_N}(x), \dots, f_{k,i}^{p_N}(x), \dots\}$ and $f_k^{p_W}(x) = \{f_{k,1}^{p_W}(x), \dots, f_{k,i}^{p_W}(x), \dots\}$ be the logits generated by narrow BL and wide BL models. Then, the loss function representing

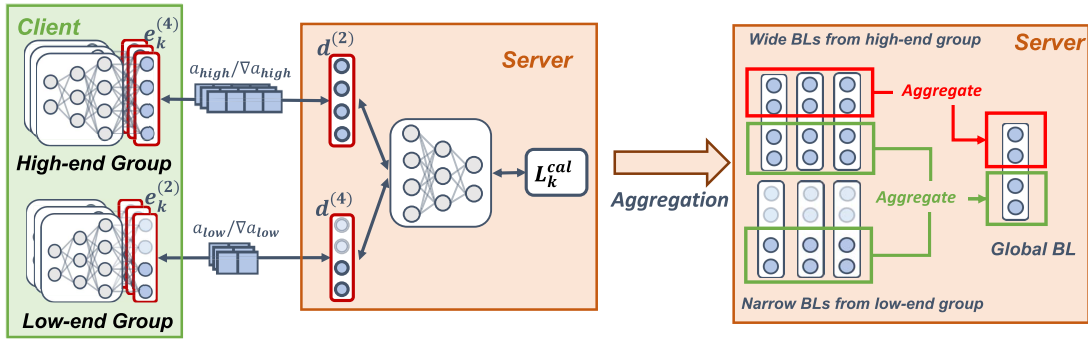


Fig. 5. In HetBL, the data of different clients are compressed by different ratios through use of different-sized BLs. The narrow BL model in low-end group of clients is a subnetwork of the wide BL model in high-end group. So during aggregation, the parameters of the narrow BL model are aggregated with the parameters of the subnetwork of the wide BL model.

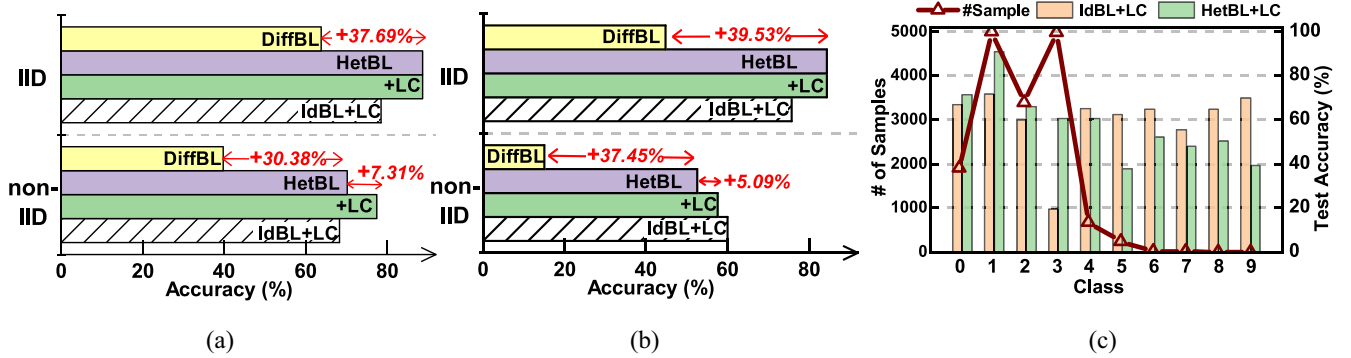


Fig. 6. Accuracy performance of identical-size BL (IdBL) and variants of different-sized BLs with 40% clients in high-end group. For different-sized BLs, the high-end group uses 16-channel BL and the low-end group uses 1-channel BL, while both groups use 1-channel BL in IdBL. (a) Dynamic group affiliation: The accuracy of HetBL is worse than IdBL even with mitigated interclient label skew using LC method (LC) [26] due to the intergroup label skew. (b) Static group affiliation: The accuracy of HetBL is worse than IdBL even with the interclient label skew is mitigated using LC (LC) [26] due to the intergroup label skew. (c) Class-wise number of samples in high-end group and class-wise accuracy of IdBL and HetBL for non-IID data. The accuracy of HetBL for underrepresented classes in high-end group is significantly lower than that of IdBL due to the intergroup label skew.

wide BL learning from narrow BL is shown as follows:

$$\mathcal{L}_k^{N2W}(f_k^{PW}(x), f_k^{PN}(x)) = - \sum_{i \in \mathcal{R}} (f_{k,i}^{PN}(x) \cdot \log f_{k,i}^{PW}(x)) - \left(\sum_{i \in \mathcal{J}} f_{k,i}^{PN}(x) \right) \cdot \log \left(\sum_{i \in \mathcal{J}} f_{k,i}^{PW}(x) \right) \quad (6)$$

where $f_{k,i}^{PN}(x)$ and $f_{k,i}^{PW}(x)$ are the softmax probabilities of class i using p_N - and p_W - channel BLs. The first term helps the wide BL model learn about underrepresented $\mathcal{R}$ classes from the narrow BL model and the second term ensures that the wide BL model does not learn about $\mathcal{J}$ classes from the narrow BL model.

Wide-to-Narrow (W2N) Knowledge Sharing: We observe that the gradients of wide and narrow BL models interfere with each other when training with one-hot label, leading to an accuracy drop, a phenomenon that is also reported in [19], [56], and [57]. To eliminate the interference, we train the narrow BL models in high-end group by distilling knowledge from the wide BL model. The loss function representing training of the narrow BL model in high-end group is as follows:

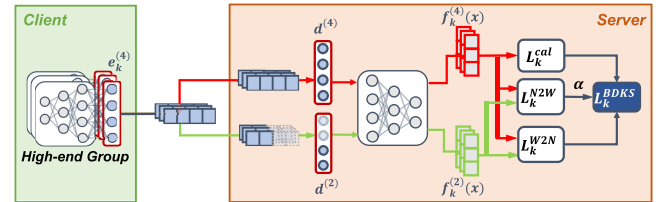


Fig. 7. BDKS is implemented in all client groups except the low-end group which has the lowest communication data rate. For simplicity, the figure shows the forward process in the high-end group. In the forward process, the activations from the high-end group is passed through the decoders corresponding to both wide ($d^{(4)}$) and narrow ($d^{(2)}$) BLs. The narrow BL model and wide BL model learn from each other through use of L_k^{N2W} and L_k^{W2N} loss functions. In the backward process, the gradients are generated by minimizing the BDKS loss L_k^{BDKS} .

$$\mathcal{L}_k^{W2N}(f_k^{PN}(x), f_k^{PW}(x)) = CE(f_k^{PN}(x), f_k^{PW}(x)) \quad (7)$$

where $CE(\cdot, \cdot)$ is the softmax cross-entropy. Since the narrow BL is a subnetwork of the wide BL, training narrow BLs individually does not introduce additional communication and computation overhead on the client side, but doubles the computation overhead at the server end.

BDKS: We propose a new BDKS loss function which combines LC and N2W knowledge sharing to train wide BL

model and W2N knowledge sharing to train narrow BL model

$$\mathcal{L}_k^{\text{BDKS}} = \underbrace{\mathcal{L}_k^{\text{cal}}(f_k^{pw}(x), y) + \alpha \cdot \mathcal{L}_k^{N2W}(f_k^{pw}(x), f_k^{pN}(x))}_{\text{training wide BL}} + \underbrace{\mathcal{L}_k^{W2N}(f_k^{pN}(x), f_k^{pw}(x))}_{\text{training narrow BL}} \quad (8)$$

where $\alpha > 0$ is the hyperparameter to control the strength of N2W knowledge distillation, which depends on the model and the dataset. To mitigate the interclient label skew, we use the loss function of LC $\mathcal{L}_k^{\text{cal}}$ here instead of cross-entropy. BDKS can also be extended to more than two groups, where the i th group uses p_i -channel BL. When the clients in the i th group send activations to the server, the server is required to train one p_i -channel BL and $i - 1$ subnetworks of BL of the p_i -channel BL as shown as follows:

$$\mathcal{L}_{k,i\text{-th}}^{\text{BDKS}} = \underbrace{\mathcal{L}_k^{\text{cal}}(f_k^{p_i}(x), y) + \alpha \cdot \mathcal{L}_k^{M2W}(f_k^{p_i}(x), f_k^{p_1}(x))}_{\text{training } p_i\text{-channel BL}} + \underbrace{\sum_{j=1}^{i-1} \mathcal{L}_k^{W2M}(f_k^{p_j}(x), f_k^{p_i}(x)) + \alpha \cdot \mathcal{L}_k^{N2M}(f_k^{p_j}(x), f_k^{p_1}(x))}_{\text{training } p_2 \text{ to the } p_{i-1}\text{th BL}} + \underbrace{\mathcal{L}_k^{M2N}(f_k^{pN}(x), f_k^{pM}(x))}_{\text{training } p_1\text{-channel BL}} \quad (9)$$

where $f_k^{p_i}(x)$ represents the logits generated by p_i -channel BL. The overhead is at the server-end which now needs to train K different sets of BLs for a system with K groups. However, increasing the number of client groups offers minimal accuracy improvement, as will be demonstrated in Section V. Note that there is no additional communication and computation overhead at the client end.

C. Algorithm-Level Description of HeteroSFL

Algorithm 1 summarizes the behavior of the clients and the server in HeteroSFL.

Clients: At the beginning of each epoch, each client first downloads the latest global parameters $\text{client}_{\text{global}}$ from the server (line 3). In each batch, the data is processed by the forward pass in the client-side model, and the resulting activations are transmitted to the server (lines 6-7). The clients then wait to receive the gradients from the server and use them to complete the backward pass followed by updating the client-side model (lines 8-10). At the end of each epoch, the updated local parameters of the client-side model are sent back to the server (line 12).

Server: At the beginning of each epoch, the server distributes the latest global parameters of the client-side model $\text{client}_{\text{global}}$ to all clients (line 18). During each batch, the server first gathers activations from all clients ($\mathbf{a}_{k \in K}$) and processes them by the forward pass using either the BDKS loss or the loss-calibration loss, depending on the clients' affiliated group (lines 20-26). After that, the server completes its backward pass and sends the resulting gradients back to clients (lines 27-28). After processing the data from K clients,

Algorithm 1 HeteroSFL client_k is the Client-Side Model of k th Client including BL; $\text{Client}_{\text{Global}}$ is the Global Client-Side Model; a_k is the Activation of the k th Client-Side Model; Grad_k is the Gradient of a_k ; (x, y) , Where x is the Sample Data and y is the Corresponding Label; $\mathbf{a}_{k \in K}$ is the List of Activations from K Clients; $\mathbf{y}_{k \in K}$ is the List of Labels from K Clients; $\text{client}_{k \in K}$ is the List of Client-Side Model from K Clients; S Indicates the Synchronization Operation Pending Until All Clients Have Completed

```

1: procedure Train_Client $_k$ :
2:   for epoch  $\in$  total_epochs do
3:      $\text{client}_{\text{global}} = \text{recieve\_params\_from\_server}()$ 
4:      $\text{client}_k = \text{client}_{\text{global}}$ 
5:     for batch  $\in$  total_batches do
6:        $a_k = \text{fwd}((x, y); \text{client}_k), (x, y) \in \mathbf{D}_k$ 
7:        $\text{send\_act\_to\_server}(a_k, y)$ 
8:        $\text{grad}_k = \text{get\_grad\_from\_server}()$ 
9:        $\text{backward}(\text{grad}_k; \text{client}_k)$ 
10:      Update  $\text{client}_k$ 
11:    end for
12:     $\text{send\_params\_to\_server}(\text{client}_k)$ 
13:  end for
14: end procedure

15:
16: procedure Train_Server:
17:   for epoch  $\in$  total_epochs do
18:      $\text{send\_params\_to\_clients}(\text{client}_{\text{global}})$ 
19:     for batch  $\in$  total_batches do
20:        $\mathbf{a}_{k \in K}, \mathbf{y}_{k \in K} = \text{get\_act\_from\_clients}()$   $\text{S}$ 
21:       for  $a_k \in \mathbf{a}_{k \in K}$  do
22:         if  $k$ th client  $\in K_{\text{high}}$  then
23:            $\text{loss}_k = \mathcal{L}_k^{\text{BDKS}}(a_k, y_k)$ 
24:         else
25:            $\text{loss}_k = \mathcal{L}_k^{\text{cal}}(a_k, y_k)$ 
26:         end if
27:          $\text{grad}_k = \text{backward}(\text{loss}_k)$ 
28:          $\text{send\_grads\_to\_client}_k(\text{grad}_k)$ 
29:       end for
30:       Update server-side model
31:     end for
32:      $\text{client}_{k \in K} = \text{get\_params\_from\_clients}()$   $\text{S}$ 
33:      $\text{client}_{\text{global}} = \text{model\_aggregation}(\text{client}_{k \in K})$ 
34:   end for
35: end procedure

36:
37: procedure model_aggregation( $\text{client}_{k \in K}$ ):
38:   Aggregating  $\text{client}_{\text{global}}$  from  $\text{client}_{k \in K}$ , where
39:   the BL is aggregated based on Eq.4 and Eq.5
40:   return  $\text{client}_{\text{global}}$ 
41: end procedure

```

the server updates the server-side model (lines 30). At the end of each epoch, the server receives the updated local parameters of the client-side model from the clients and aggregates them to generate the global parameters (lines 32-33).

TABLE III
EXPERIMENTAL SETUP

	Static Affiliation	Dynamic Affiliation
Number of clients	10 and 50	50
Number of groups	2 and 3	3
Data distribution	IID and non-IID with $\lambda = 0.05$	
Data rate	16 : 1 for 2 groups; 16 : 8 : 1 for 3 groups	
Client-side Model	3 convolution layers for VGG11; 5 residual blocks for ResNet20	

V. EXPERIMENTAL RESULTS

In this section, we implement *HeteroSFL* for static and dynamic group affiliation cases to demonstrate that the proposed algorithms can achieve significant training-time reduction with minimum accuracy loss for both IID and non-IID data. We also study the effectiveness of BDKS in improving the performance of HeteroSFL. All these experiments have been conducted on a Desktop server with Ubuntu desktop 16.04-64bit and equipped with an NVIDIA Geforce RTX 3090 GPU with 24 GB memory and an Intel Core i7-12700K CPU, 3.6 GHz frequency with 32 GB memory. We use Pytorch 2.4.0 and CUDA 11.8.

A. Experimental Setup

The detailed experimental setup is summarized in Table III. *Datasets and Models:* We choose VGG11 and ResNet20 as the representative models since they are widely used in FL and SFL studies [6], [18], [26], [53], [58], [59]. Specifically we use VGG11 on CIFAR-10 and ResNet20 on CIFAR-100. For VGG11, the model is split such that the client-side model has 3 convolution layers and for ResNet20 the client-side model has 5 residual blocks. This split is shown to keep the privacy of clients [58]. We also provide results for two other layer-splitting settings in Appendix B. To emulate different degrees of interclient label skew, the datasets are partitioned with a commonly used Latent Dirichlet Sampling [60] with parameter λ , where lower λ represents data with a higher non-IID degree.

Heterogeneous Clients With Different-Sized BLs: The heterogeneous clients are separated into multiple groups and unless mentioned specifically, the clients are mapped to groups randomly as in prior works [27], [61]. We show results with 10 and 50 clients. The data rate difference between different groups of clients ranges from 4:1 to 16:1, which is reasonable given the study in [62]. The number of channels in the activations generated by the client-side model is 256 for VGG11 and 32 for ResNet20. We sweep the BL size from 1–256 channels for VGG11 and 1–32 for ResNet20 to study the performance of HeteroSFL for different communication time reduction.

Competing Methods: We compare the performance of HeteroSFL with baseline SFL [6] and several other competing methods. SR reduces the training time by reducing the number of samples processed by clients per epoch. Top-k sparsity [8] reduces the training time by sending the activations with top-k magnitude to server. IdBL [11] uses IdBLs in both high- and low-end groups. We also compare HeteroSFL with a variant

TABLE IV

ACCURACY AS A FUNCTION OF TRAINING TIME REDUCTION AND DIFFERENT PROPORTION OF CLIENTS IN HIGH-END GROUP WITH $\lambda = 0.05$ NON-IID DATA FOR VGG11 WITH 10 CLIENTS IN STATIC GROUP AFFILIATION. THE DATA RATE RATIO BETWEEN CLIENTS IN HIGH-END AND LOW-END GROUP IS 16:1. BASELINE SFL HAS AN ACCURACY OF 70.69% WITH NO REDUCTION IN TRAINING TIME

Time Redn.	256×		128×		64×	
High-end	30%	60%	30%	60%	30%	60%
SR	17.66	29.18	35.63	40.45	42.84	55.02
Top-k	53.97	57.43	56.60	62.65	61.01	66.60
IdBL	60.07	60.07	66.20	66.20	68.82	68.82
IdBL+dropout	36.85	56.98	39.11	59.19	38.98	58.88
HetBL	53.28	60.95	56.99	61.87	60.34	63.50
HeteroSFL	61.13	65.10	67.80	69.19	68.52	69.45

of IdBL, IdBL+channel-wise Dropout (IdBL+dropout) [63], where the high-end and low-end clients use IdBL, and the client picks up p channels based on the magnitude of activations of different channels and zeroes out the other channels in every epoch. We implement LC [26] for all competing methods, except baseline SFL.

Hyperparameters. The key parameters affecting the performance of HeteroSFL are the two BDKS hyperparameters, namely, α and θ , and the LC hyperparameter τ . The hyperparameter α controls the magnitude of N2W knowledge sharing; higher α means wide-BL learns more from narrow-BL. α is a function of the model and dataset, and we sweep α from 0.5 to 5 with step of 0.5 to find the best value. θ is the percentage threshold of underrepresented class. We set $\theta = 10\%$ since when high-end group contains less than 10% of samples of a specific class, the accuracy of the class using the wide-BL model drops significantly. We tune the LC hyperparameter τ by sweeping from 0 to 10 in increments of 1 and found that $\tau = 5$ provides satisfactory performance in most experiments. Therefore, we set $\tau = 5$ for all experiments. We do not specifically tune the hyperparameters of the SGD optimizer including the learning rate, momentum, and weight decay but use the same values for all experiments for fair comparison. For detailed hyperparameter settings, please see Table XI in the Appendix.

B. HeteroSFL for Static Group Affiliation

For ease of analysis, we first assign the clients to two groups - high-end and low-end in static group affiliation case and then extend the method to handle more groups.

HeteroSFL Achieves the Best Accuracy: Table IV compares the accuracy of HeteroSFL with all other competing methods as a function of training time reduction and proportion of clients in high-end group for non-IID data with $\lambda = 0.05$ data. The ratio of data rates between high-end and low-end groups of clients is 16:1. Based on the data rate ratios, to reduce the training time by 256×, the communication data of low-end group is reduced by 256×, while the high-end group is reduced only by 16×. For the same training time reduction (256×, 128× and 64×) and proportion of clients (30% and 60%) in high-end group, the proposed BL-based HeteroSFL achieves 30.07% and 7.16% higher accuracy, on average, compared

TABLE V
ACCURACY AS A FUNCTION OF PROPORTION OF CLIENTS IN HIGH-END GROUP FOR RESNET20 WITH 10 CLIENTS. THE TRAINING TIME REDUCTION IS $32\times$ AND THE DATA DISTRIBUTION IS NON-IID WITH $\lambda = 0.05$

High-end	SR	Top-k	IdBL	IdBL +dropout	HetBL	HeteroSFL
30%	45.86	47.29	50.44	8.53	46.66	52.96
60%	49.25	50.63	50.44	5.67	54.23	54.54

TABLE VI
ACCURACY AS A FUNCTION OF TRAINING TIME REDUCTION WITH IID DATA FOR VGG11 WITH 10 CLIENTS. THE PROPORTION OF CLIENTS IN HIGH-END GROUP IS 60%

	SR	Top-k	IdBL	IdBL +dropout	HetBL	HeteroSFL
64 $\times$	64.91	85.34	86.50	67.85	85.96	87.61
128 $\times$	64.42	83.77	83.42	65.41	86.78	86.63
256 $\times$	64.44	81.67	77.88	60.83	87.98	88.87

with SR and Top-k, respectively. This indicates that using BL to compress the activations and gradients still results in the lowest accuracy drop in a heterogeneous system. When compared to IdBL, HeteroSFL also achieves higher accuracy, demonstrating that compressing communication data differently for different clients can significantly improve performance without increasing the training time. The benefit of HeteroSFL over IdBL becomes more significant when the proportion of high-end groups increases since more high-end clients using wide BL allowing for more information exchange between client and server. The highest gain over IdBL is with 60% clients in high-end group and training time reduction of $256\times$. Compared with HeteroSFL, IdBL+dropout achieves 18.39% lower accuracy, on average, and consistently performs worse than IdBL, indicating that training different sets of channels each time fails to address the intergroup label skew.

Table V shows the performance of HeteroSFL with $32\times$ training time reduction for ResNet20 with 10 clients. We see that HeteroSFL achieves the best accuracy among all competing methods. Also, the performance is better when the proportion of clients in high-end group is large, as expected. Overall, the trend in performance is similar to that in VGG11.

Table VI shows the performance of HeteroSFL as a function of training time reduction for VGG11 with IID data. HeteroSFL still achieves the best accuracy compared with all competing methods, which is 23.11%, 4.11%, 5.10%, and 23.01% higher than SR, Top-k, IdBL, and IdBL+dropout on average, respectively.

BDKS Improves the Accuracy of Underrepresented Classes: The non-IID data distributions in the static group affiliation case causes the intergroup label skew. In such cases, simply using HetBL results in lower accuracy than IdBL, and the gap between HetBL and IdBL becomes larger when there are fewer clients in high-end groups, as shown in Tables IV and V. By implementing BDKS on top of HetBL (a.k.a HeteroSFL), the accuracy is improved significantly over HetBL and beats IdBL. When the data distribution is IID, the intergroup label skew does not exist anymore, and HeteroSFL achieves the same results as HetBL, as shown in Table VI. To investigate

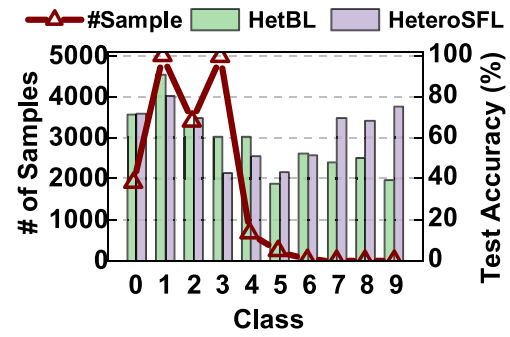


Fig. 8. Class-wise accuracy of HetBL and HeteroSFL with 40% high-end group for 10 clients.

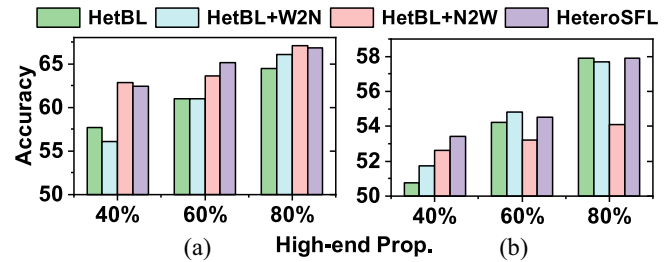


Fig. 9. Accuracy of HeteroSFL and HetBL with W2N or N2W knowledge sharing for (a) VGG11 and (b) ResNet20 with 10 clients and $\lambda = 0.05$ non-IID data. The proportion of clients in high-end group is varied from 40% to 80%. The training time reduction is $256\times$ for VGG11 and $32\times$ for ResNet20.

the reason for the accuracy improvement due to BDKS, the class-wise accuracy of HeteroSFL is compared with HetBL. The setting in Fig. 8 is the same as in Fig. 6(c). The accuracy of underrepresented classes (i.e., classes 5 to 9) increases significantly, indicating that BDKS effectively alleviates the overfitting of the wide BL model.

Effectiveness of N2W and W2N in BDKS: Next, we explore the effectiveness of the two types of knowledge sharing in BDKS. We consider the effect of only N2W in HetBL+N2W and the effect of only W2N in HetBL+W2N; the corresponding results for VGG11 and ResNet20 are shown in Fig. 9. When the number of high-end clients is less than 40%, HetBL+N2W achieves higher accuracy than HetBL, while HetBL+W2N does not significantly improve the accuracy. In such a scenario, the intergroup label skew is a serious problem due to few clients in the high-end group, and N2W is key to address the intergroup label skew. When the proportion of high-end clients is larger than 80%, there is almost no intergroup label skew and W2N begins playing a role. The results illustrate that both N2W and W2N knowledge sharing is necessary for HeteroSFL to achieve higher accuracy, and their role is different based on the proportion of high-end clients.

Scalability: We analyze the performance of HeteroSFL with larger number of groups. Table VII shows the performance of HeteroSFL for VGG11 and ResNet20 when there are three groups of clients for non-IID data with $\lambda = 0.05$. For VGG11, for $256\times$ training time reduction, HeteroSFL achieves up to 41.48%, 7.06%, and 6.72% higher accuracy compared with SR, Top-k Sparsity, and IdBL.

TABLE VII
ACCURACY OF HETERO-SFL FOR THREE GROUPS AND $\lambda = 0.05$ NON-IID DATA. THE NUMBER OF CLIENTS IS 10 AND THE TRAINING TIME REDUCTION IS $256\times$. THE DATA RATE RATIO FOR THE THREE GROUPS IS 1:8:16

	Proportion of clients	SR	Top-k Sparsity	IdBL	HetBL	HeteroSFL
VGG11-CIFAR10	60%, 20%, 20%	20.31	53.69	60.07	49.09	60.75
	20%, 40%, 40%	22.90	58.72	60.07	61.66	64.08
	20%, 20%, 60%	30.19	60.50	60.07	63.47	66.79
ResNet20-CIFAR100	60%, 20%, 20%	46.76	46.32	50.44	48.35	52.13
	20%, 40%, 40%	53.31	50.80	50.44	54.72	55.54
	20%, 20%, 60%	54.49	53.43	50.44	56.96	56.33

TABLE VIII
ACCURACY OF HETBL AND HETERO-SFL WITH 50 CLIENTS WHEN THE TRAINING TIME REDUCTION IS $256\times$ AND THE DATA RATE RATIO BETWEEN HIGH-END AND LOW-END GROUP OF CLIENTS IS 16:1

# of clients	High-end group	HetBL	HeteroSFL
50-clients (Data-device uncorrelated)	30%	57.15	60.92 (+3.77)
	60%	61.43	63.34 (+1.91)
50-clients (Data-device correlated)	30%	53.22	59.16 (+5.94)
	60%	61.38	64.35 (+2.98)

TABLE IX
ACCURACY AS A FUNCTION OF TRAINING TIME REDUCTION AND DATA DISTRIBUTION FOR VGG11 WITH 50 CLIENTS IN DYNAMIC GROUP AFFILIATION CASE. THE 50 CLIENTS ARE CLUSTERED INTO THREE GROUPS WITH DATA RATE RATIOS OF 1:8:16

Data Dist.	IID			Non-IID $\lambda = 0.05$		
Time Redn.	$256\times$	$128\times$	$64\times$	$256\times$	$128\times$	$64\times$
SR	25.02	61.72	84.85	20.25	31.40	67.64
Top-k	76.41	82.80	85.96	65.00	70.34	73.65
IdBL	72.56	77.84	83.53	61.27	69.74	72.92
HetBL	85.65	85.67	86.04	74.01	73.92	74.66
HeteroSFL	85.34	84.28	84.96	72.63	73.84	73.95

Next we consider the effect of number of client groups N on the accuracy performance. If $N = 1$, all clients are clustered to use 1-channel BL. This is equivalent to the IdBL method with accuracy of 60.07%, as shown in Table IV. Now if $N = 2$, (1-channel BL for low-end and medium-end clients, and 16-channel BL for high-end clients), the accuracy is 65.10%, which is 5.03% higher than the $N = 1$ case. If $N = 3$ (1-channel BL for low-end, 4-channel for medium-end, and 16-channel BL for high-end), we see that the accuracy is 66.79%, which is only 1.69% higher than the accuracy for the $N = 2$ case. Similarly, for ResNet20, the accuracy improves by 4.1% from 50.44% to 54.54% when N increases from 1 to 2 (as shown in Table V). However, the accuracy gain is just 1%, reaching 55.54%, when N increases from 2 to 3 (as shown in Table VII). Thus, the accuracy improvement from $N = 2$ to $N = 3$ is significantly smaller compared to the improvement from $N = 1$ to $N = 2$. Since the overhead in the server side increases as N increases, we suggest that when there are multiple types of heterogeneous clients, the clients should be clustered into $N = 2$ or 3 groups based on their communication data rates.

Next we analyze the performance when the number of clients increases. Table VIII shows the performance of HeteroSFL and HetBL for 50 clients and non-IID data distribution with $\lambda = 0.05$. Two different client-group mapping policies are used. The first mapping randomly maps the clients to groups, leading to low degree of intergroup label skew. In contrast, the second mapping maps clients with similar label distributions to the same group, leading to high degree of intergroup label skewness. In the second mapping, the data label distribution and client heterogeneity are correlated, as was reported in [12]. In the uncorrelated case with a low level of intergroup label skew, BDKS helps improve accuracy by 3.77% over HetBL. For the correlated case with a higher

degree of intergroup label skew, BDKS improves accuracy even more by 5.94%, as shown in Table VIII.

Different Client-Side Model: To further validate the effectiveness of HeteroSFL, we evaluate its performance using client-side models of varying sizes—specifically, VGG11 with 2 and 4 convolutional layers and ResNet20 with 3 and 4 residual blocks. Detailed results are provided in Appendix B. HeteroSFL consistently achieves the highest accuracy among all methods, with the accuracy improvement becoming more significant as the depth of the client-side model increases. This is because HeteroSFL can effectively address the adverse effect that non-IID data has on deeper client-side models.

C. HetBL for Dynamic Group Affiliation Case

In the dynamic group affiliation case, all clients use the same communication protocol, but the clients can be assigned to different groups in different epochs due to the time-varying transmission channel condition. The 50 clients in SFL system are distributed across three different groups with data rate ratios of 1:8:16. Since the different clients are controlled by different base stations, the bandwidth B_i^t , transmission power P_i^t and transmission channel conditions h_i^t of clients can be different from each other. We evaluate the case when a client's group assignment follows a uniform distribution during a specific epoch. Table IX shows the results for VGG11.

HetBL and HeteroSFL Achieve the Best Accuracy: Due to the group affiliation of clients changing over time, there is no intergroup label skewness problem in the dynamic group affiliation case and the proposed HetBL and HeteroSFL methods have similar accuracy. They also consistently achieve the highest accuracy across different training time reduction and data distributions. With $256\times$ communication reduction, the proposed HetBL outperforms SR and Top-K sparsity

TABLE X
ACCURACY AS A FUNCTION OF DATA DISTRIBUTIONS FOR RESNET20
WITH 50 CLIENTS IN DYNAMIC GROUP AFFILIATION. THE TRAINING
TIME REDUCTION IS $32\times$

	SR	Top-k	IdBL	HetBL	Hetero-SFL
IID	58.38	57.56	53.03	60.79	59.58
Non-IID $\lambda = 0.05$	56.94	54.72	50.52	58.39	57.99

by 60.63% and 9.24% for IID data, and by 54.26% and 9.51% for non-IID data. Compared with IdBL, HetBL achieves 13.09% and 13.24% higher accuracy for IID and non-IID data, respectively. When the training time reduces more, the gap between HetBL and SR and Top-k becomes larger since SR and Top-k lose more important information.

Table X shows the performance of HetBL as a function of data distribution for ResNet20 with $32\times$ training time reduction. HetBL and HeteroSFL achieve the best accuracy among all competing methods, as in VGG11.

VI. CONCLUSION

In this article, we consider SFL with heterogeneous clients where the heterogeneity is with respect to communication data rates of clients and distribution of data processed by these clients. We study two cases: 1) dynamic group affiliation, where the clients in a group change over time and 2) static group affiliation, where the clients in a group do not change. We first design HetBL to reduce the training time by compressing the transmitted data in high-end and low-end groups differently through use of different-sized BLs. While this is sufficient to improve the accuracy compared to IdBL for the dynamic group affiliation case, it is not so for the static group affiliation case due to intergroup label skew. So we propose BDKS where the wide BL model learns underrepresented classes from narrow BL to address the intergroup label skew and the narrow BL model learns the representative classes from wide BL to improve its performance. We combine HetBL with BDKS and build *HeteroSFL*, the first SFL framework with heterogeneous clients that process non-IID data characterized by interclient and intergroup label skew. For the static group affiliation, when there is significant label distribution skew, HeteroSFL can reduce the training time of SFL by $16\times$ to $256\times$ with 1.24% to 5.07% accuracy loss compared to naive SFL, on average, for VGG11 on CIFAR10 data.

In this work, we currently only consider the data heterogeneity and the communication heterogeneity across clients. However, the clients can be heterogeneous with respect to their computation capability [18], [27] and/or battery capacity [64], [65]. Computation heterogeneity cannot be ignored especially when the clients have comparable communication and computation time. We plan to address the heterogeneity in computation capability by using different-sized client-side models through use of different cut layers [66], [67] or model pruning similar to FL schemes [18], [27]. Our future work will explore SFL performance tradeoffs under computation, communication, battery energy, and data heterogeneity.

APPENDIX

Configurations and Simulations: The training process of the SFL system is simulated in PyTorch on a single machine initialized with multiple client-side models and a server-side model. During training, the machine first calls the *.forward()* method of client-side models to generate the activations one by one, to simulate the in-parallel forward propagation of clients. Then, the *.forward()* method of the server-side model is called to process all the activations at one time. Finally, we call *loss.backward()* to run the backward propagation for the server-side model and all client-side models. The communication time is manually computed based on the data rates and the communication data size.

Hyperparameters: The detailed experimental settings are as follows and summarized in Table XI.

- 1) *Epoch:* Every experiment is run for 200 epochs.
- 2) *Batch size:* For a 10-client SFL system, the batch size is 13 samples per client and so $10 \times 13 = 130$ samples per batch on the server side; For 50-client SFL system, the batch size is 3 samples per client and so $50 \times 3 = 150$ samples per batch on the server side.
- 3) *Learning rate:* The learning rate for server-side model is 0.01 for VGG11 and 0.1 for ResNet20. The learning rate for client-side model is 0.02 for both VGG11 and ResNet20. The learning rate is reduced during training using the *CosineAnnealingLR* scheduler in Pytorch.
- 4) *Optimizer:* SGD is used as the optimizer with momentum = 0.9, weight_decay = $5e-4$.
- 5) *τ in LC [26]:* τ is the hyperparameter for LC. It is tuned in the range 0 to 10; the final value is set as 5.
- 6) *θ in Section IV-B:* We define θ to be the percentage of samples in an underrepresented class compared to that in the global dataset. $\theta = 10\%$ in our experiments.
- 7) *α in (8):* α controls the magnitude of N2W knowledge sharing and is a function of the model and dataset. Specifically, the best value of α changes when the client-side model and the number of BL channel changes; but does not change with the number of clients, and the non-IID degree of data distribution. We list the α values for the different configurations used in the experiments as follows.

a) *VGG11 With Different Client-Side Models:*

- i) *2 Convolution Layers:* α is set as 4.0 when using 1-channel narrow BL and 16-channel wide BL.
- ii) *3 Convolution Layers:* α is set as 3.2, 2.8 and 4.0 when using 4-, 2-, and 1-channel narrow BL, respectively, for 16-channel wide BL. α is set as 3 when using 8- and 4-channel wide BL for 1-channel narrow BL.
- iii) *4 Convolution Layers:* α is set as 3.5 when using 2-channel narrow BL and 16-channel wide BL.
- iv) *Multiple Groups:* α is set as 2.6 and 0.5 when clients are separated into three and four groups, respectively.

TABLE XI
HYPERPARAMETER TUNING PROCESS AND VALUES

Hyper-parameters	Value
Epoch	200
Batch Size	130 for 10 clients; 150 for 50 clients;
Learning Rate	Server-side model: - 0.01 for VGG11 - 0.1 for ResNet20 Client-side model: 0.02
Momentum	0.9
Weight Decay	5e-4
τ controls the magnitude of the pairwise label margin in logit calibration	5 (tuned in range (0, 10])
θ controls the percentage threshold of the underrepresented class in BDKS	10%
α controls the magnitude of N2W knowledge sharing in BDKS	4.0 for VGG11; 1.5 for ResNet20; (tuned in range [0.5, 5])

b) *ResNet20 With Different Client-Side Models:*

- i) *3 Residual Blocks:* α is set as 1.5 when using 1-channel narrow BL for 16-channel wide BL.
- ii) *4 Residual Blocks:* α is set as 2.0 when using 2-channel narrow BL for 16-channel wide BL.
- iii) *5 Residual Blocks:* α is set as 1.4, 1.3 and 0.5 when using 4-, 2-, and 1-channel narrow BL, respectively, for 16-channel wide BL.

A. *Feature Distribution Skew*

We also consider feature distribution skew as another example of non-IID data distribution. We use ResNet20 model and the Office Caltech dataset with 10 classes from 4 different domains, WebCam, Amazon, Caltech, and Dslr. We split the data from 4 domains to 4 clients (one domain per client). The high-end clients use wide-BL with 16 channels and the low-end clients use narrow-BL with 1 channel. Since there is no existing method that handles feature skew in SFL, we compare our results with federated prototypes learning (FPL) [68], an FL paper that handles domain shift. The results of FPL and HeteroSFL under static group affiliation are shown in Table XII. The results show that even though HeteroSFL was not designed to handle feature distribution skew like FPL, it can handle feature distribution skew with only a small drop in accuracy compared to FPL. We believe that HeteroSFL's good performance is due to BDKS where the wide-BL model is able to learn from the underrepresented domains seen by the narrow-BL model, thereby mitigating feature distribution skew.

B. *Different Client-Side Models*

In Tables IV and V, we evaluate HeteroSFL using VGG11 with 3 convolution layers, and ResNet20 with 5 residual blocks on the client side. To further demonstrate the effectiveness of HeteroSFL, Table XIII presents the accuracy comparisons of HeteroSFL and other methods where the client-side models

TABLE XII
ACCURACY COMPARISONS OF HETEROSFL AND FPL FOR RESNET20 WITH CALTECH. BOTH FL AND SFL HAVE FOUR CLIENTS. THE SFL SYSTEM IS STATIC GROUP AFFILIATION WHERE 50% OF HIGH-END CLIENTS USE WIDE-BL WITH 16 CHANNELS AND THE OTHER 50% OF LOW-END CLIENTS USE NARROW-BL WITH 1 CHANNEL

Domain Name	WebCam	Amazon	Caltech	Dslr	Avg.
FPL [68]	86.21	94.21	86.61	96.67	90.92
HeteroSFL	87.93	87.37	86.16	96.67	89.52

TABLE XIII
ACCURACY PERFORMANCE OF SFL USING IDBL, TOP-K SPARSITY, AND HETEROSFL WITH DIFFERENT CLIENT-SIDE MODELS UNDER $\lambda = 0.05$ FOR VGG11 ON CIFAR10 AND RESNET20 ON CIFAR100. THE TRAINING TIME REDUCTION IS 128 $\times$. THE DATA RATE RATIO BETWEEN HIGH-END AND LOW-END GROUPS IS 16:1 WITH 30% HIGH-END CLIENTS

Models	VGG11		ResNet20	
Client-side Model	2 conv.	4 conv.	3 blocks	4 blocks
Top-k	55.82	47.12	54.21	53.85
IdBL	65.89	53.94	58.69	56.18
HeteroSFL	66.90	56.56	59.00	57.28

have 2 and 4 convolution layers for VGG11 and 3 and 4 residual blocks for ResNet20 with 128 $\times$ training time reduction and $\lambda = 0.5$ non-IID data distribution.

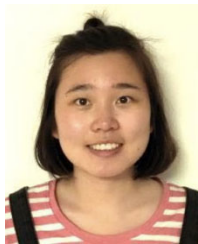
Notably, the accuracy improvement becomes more pronounced as the client-side model depth increases. With the same proportion of high-end clients, HeteroSFL improves accuracy over IdBL by 1.01% and 2.62% for VGG11 with 2 and 4 convolutional layers, respectively. Similarly, for ResNet20, accuracy increases by 0.31% and 0.90% for 3 and 4 residual blocks, with an even larger gain of 2.52% for 5 residual blocks (as shown in Table V). This trend suggests that the performance degradation due to non-IID data in deeper client-side models is successfully addressed by HeteroSFL.

REFERENCES

- [1] V. Ghenescu, R. E. Mihaescu, S.-V. Carata, M. T. Ghenescu, E. Barnoviciu, and M. Chindea, "Face detection and recognition based on general purpose DNN object detector," in *Proc. Int. Symp. Electron. Telecommun. (ISETC)*, 2018, pp. 1–4.
- [2] S. Park, G. Kim, J. Kim, B. Kim, and J. C. Ye, "Federated split vision transformer for COVID-19CXr diagnosis using task-agnostic training," 2021, *arXiv:2111.01338*.
- [3] Y. Liu, J. James, J. Kang, D. Niyato, and S. Zhang, "Privacy-preserving traffic flow prediction: A federated learning approach," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7751–7763, Aug. 2020.
- [4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [5] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *J. Netw. Comput. Appl.*, vol. 116, pp. 1–8, Aug. 2018.
- [6] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8485–8493.
- [7] J. Wang et al., "FedLite: A scalable approach for federated learning on resource-constrained clients," 2022, *arXiv:2201.11865*.
- [8] L. Liu et al., "Dynamic sparse graph for efficient deep learning," 2018, *arXiv:1810.00859*.
- [9] X. Chen, J. Li, and C. Chakrabarti, "Communication and computation reduction for split learning using asynchronous training," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, 2021, pp. 76–81.

- [10] A. E. Eshratifar, A. Esmaili, and M. Pedram, "BottleNet: A deep learning architecture for intelligent mobile cloud computing services," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, 2019, pp. 1–6.
- [11] A. Ayad, M. Renner, and A. Schmeink, "Improving the communication and computation efficiency of split learning for IoT applications," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2021, pp. 1–6.
- [12] K. Maeng, H. Lu, L. Melis, J. Nguyen, M. Rabbat, and C.-J. Wu, "Towards fair federated recommendation learning: Characterizing the inter-dependence of system and data heterogeneity," in *Proc. 16th ACM Conf. Recommender Syst.*, 2022, pp. 156–167.
- [13] J. Jiang, B. Kantarci, S. Oktug, and T. Soyata, "Federated learning in smart city sensing: Challenges and opportunities," *Sensors*, vol. 20, no. 6230, pp. 20889–20901, 2020.
- [14] T. Zheng, A. Li, Z. Chen, H. Wang, and J. Luo, "Autofed: Heterogeneity-aware federated multimodal learning for robust autonomous driving," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, 2023, pp. 1–15.
- [15] D. C. Nguyen et al., "Federated learning for smart healthcare: A survey," *ACM Comput. Surveys*, vol. 55, no. 3, pp. 1–37, 2022.
- [16] Y. Gao et al., "End-to-end evaluation of federated learning and split learning for Internet of Things," in *Proc. Int. Symp. Reliable Distrib. Syst. (SRDS)*, 2020, pp. 91–100.
- [17] R. Chen et al., "EEFL: High-speed wireless communications inspired energy efficient federated learning over mobile devices," in *Proc. 21st Annu. Int. Conf. Mobile Syst., Appl. Services*, 2023, pp. 544–556.
- [18] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–24.
- [19] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. Venieris, and N. Lane, "FjORD: Fair and accurate federated learning under heterogeneous targets with ordered dropout," in *Proc. 35th Conf. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 12876–12889.
- [20] Z. Zhu, J. Hong, S. Drew, and J. Zhou, "Resilient and communication efficient learning for heterogeneous federated systems," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 27504–27526.
- [21] D. Makhija, X. Han, N. Ho, and J. Ghosh, "Architecture agnostic federated learning for neural networks," 2022, *arXiv:2202.07757*.
- [22] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 2351–2363.
- [23] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, 2021.
- [24] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [25] Z. Tang, Y. Zhang, S. Shi, X. He, B. Han, and X. Chu, "Virtual homogeneity learning: Defending against data heterogeneity in federated learning," 2022, *arXiv:2206.02465*.
- [26] J. Zhang et al., "Federated learning with label distribution skew via logits calibration," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 26311–26329.
- [27] F. Ilhan, G. Su, and L. Liu, "ScaleFL: Resource-adaptive federated learning with heterogeneous clients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24532–24541.
- [28] J. Li, L. Lyu, D. Iso, C. Chakrabarti, and M. Spranger, "MocoSFL: Enabling cross-client collaborative self-supervised learning," in *Proc. 11th Int. Conf. Learn. Represent., (ICLR)*, 2023, pp. 1–12.
- [29] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Wang, and C. Qiao, "Accelerating federated learning with data and model parallelism in edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 904–918, Feb. 2024.
- [30] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Huang, and C. Qiao, "ParallelSFL: A novel split federated learning framework tackling heterogeneity issues," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, 2024, pp. 845–860.
- [31] Y. Gao, B. Hu, M. B. Mashhadi, W. Wang, and M. Bennis, "PipeSFL: A fine-grained parallelization framework for split federated learning on heterogeneous clients," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1774–1791, Mar. 2025.
- [32] C. He, M. Annamaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large CNNs at the edge," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 14068–14080.
- [33] D.-J. Han, H. I. Bhatti, J. Lee, and J. Moon, "Accelerating federated learning with split learning on locally generated losses," in *Proc. ICML Workshop Federated Learn. User Privacy Data Confidential. ICML Board*, 2021, pp. 1–12.
- [34] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.
- [35] Z. Lian, J. Cao, Z. Zhu, X. Zhou, and W. Liu, "GOFL: An accurate and efficient federated learning framework based on gradient optimization in heterogeneous IoT systems," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 12459–12474, Apr. 2024.
- [36] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2019, pp. 1–7.
- [37] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, "FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients," in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Services*, 2022, pp. 436–449.
- [38] Z. Wang et al., "Asynchronous federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6961–6978, Sep. 2022.
- [39] Y. Chen, Y. Ning, M. Slawski, and H. Rangwala, "Asynchronous online federated learning for edge devices with non-IID data," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2021, pp. 15–24.
- [40] M. Morafah, V. Kungurtsev, H. Chang, C. Chen, and B. Lin, "Towards diverse device heterogeneous federated learning via task arithmetic knowledge integration," 2024, *arXiv:2409.18461*.
- [41] J. Han, W. Ni, and L. Li, "Semi-federated learning for connected intelligence with computing-heterogeneous devices," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34078–34092, Nov. 2024.
- [42] R. Lee et al., "Recurrent early exits for federated learning with heterogeneous clients," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 26568–26588.
- [43] S. Alam, L. Liu, M. Yan, and M. Zhang, "FedRolex: Model-heterogeneous federated learning with rolling sub-model extraction," in *Proc. 36th Conf. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 29677–29690.
- [44] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, 2022, pp. 965–978.
- [45] Y. Cai and T. Wei, "Efficient split learning with non-IID data," in *Proc. 23rd IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, pp. 128–136.
- [46] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6357–6368.
- [47] L. Gao, H. Fu, L. Li, Y. Chen, M. Xu, and C.-Z. Xu, "FedDC: Federated learning with non-IID data via local drift decoupling and correction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10112–10121.
- [48] X. Zhou, X. Lei, C. Yang, Y. Shi, X. Zhang, and J. Shi, "Handling data heterogeneity for IoT devices in federated learning: A knowledge fusion approach," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8090–8104, Mar. 2024.
- [49] W. Han, J. Peng, J. Yu, J. Kang, J. Lu, and D. Niyato, "Heterogeneous data-aware federated learning for intrusion detection systems via Meta-sampling in artificial intelligence of things," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13340–13354, Apr. 2024.
- [50] C. Xu, Z. Hong, M. Huang, and T. Jiang, "Acceleration of federated learning with alleviated forgetting in local training," 2022, *arXiv:2203.02645*.
- [51] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12878–12889.
- [52] H. Chen, A. Frikha, D. Krompass, and V. Tresp, "FRAug: Tackling federated learning with non-IID features via representation augmentation," 2022, *arXiv:2205.14900*.
- [53] C.-Y. Huang, K. Srinivas, X. Zhang, and X. Li, "Overcoming data and model heterogeneities in Decentralized federated learning via synthetic anchors," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 20111–20133.
- [54] Z. Yang et al., "FedFed: Feature distillation against data heterogeneity in federated learning," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 60397–60428.
- [55] H. Chen, S. Huang, D. Zhang, M. Xiao, M. Skoglund, and H. V. Poor, "Federated learning over wireless IoT networks with optimized communication and resources," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 16592–16605, Sep. 2022.
- [56] A. Mohtashami, M. Jaggi, and S. Stich, "Masked training of neural networks with partial gradients," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 5876–5890.
- [57] H. Baek et al., "Joint superposition coding and training for federated learning over multi-width neural networks," in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, 2022, pp. 1729–1738.

- [58] J. Li, A. S. Rakin, X. Chen, Z. He, D. Fan, and C. Chakrabarti, "ResSFL: A resistance transfer framework for defending model inversion attack in split federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10194–10202.
- [59] L. Qu et al., "Rethinking architecture design for tackling data heterogeneity in federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10061–10071.
- [60] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
- [61] K. Zhang, Y. Dai, H. Wang, E. Xing, X. Chen, and L. Sun, "Memory-adaptive depth-wise heterogeneous federated learning," 2023, *arXiv:2303.04887*.
- [62] SpeedTest. "Global median speeds." Aug. 2023. [Online]. Available: <https://www.speedtest.net/global-index>.
- [63] S. Hou and Z. Wang, "Weighted channel dropout for regularization of deep convolutional neural network," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 8425–8432.
- [64] C. Yang et al., "Characterizing impacts of heterogeneity in federated learning upon large-scale smartphone data," in *Proc. Web Conf.*, 2021, pp. 935–946.
- [65] X. Chen, J. Li, and C. Chakrabarti, "Energy and loss-aware selective updating for SplitFed learning with energy harvesting-powered devices," *J. Signal Process. Syst.*, vol. 94, no. 10, pp. 961–975, 2022.
- [66] D. Wu, R. Ullah, P. Harvey, P. Kilpatrick, I. Spence, and B. Varghese, "FedAdapt: Adaptive offloading for IoT devices in federated learning," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 20889–20901, Nov. 2022.
- [67] Z. Lin, G. Qu, W. Wei, X. Chen, and K. K. Leung, "AdaptSFL: Adaptive split federated learning in resource-constrained edge networks," 2024, *arXiv:2403.13101*.
- [68] W. Huang, M. Ye, Z. Shi, H. Li, and B. Du, "Rethinking federated learning with domain shift: A prototype view," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 16312–16322.



Xing Chen received the Ph.D. degree in electrical engineering from the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA, in 2023.

Her research interests include systems for algorithms and architectures for machine learning, federated learning, and heterogeneous SoC systems.



Jingtao Li (Member, IEEE) received the Ph.D. degree in electrical engineering from the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA, in 2023.

He is currently a Research Scientist with Sony AI, New York City, NY, USA. His research interest lies in federated learning with focus on its practicality and utility in challenging real-world applications.

Dr. Li is the recipient of the Outstanding Doctoral Dissertation Award from the Fulton Schools of Engineering.



Deliang Fan (Senior Member, IEEE) received the M.S. and Ph.D. degrees in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2012 and 2015, respectively.

He is currently an Associate Professor with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA. His primary research interests include AI hardware, digital chip design, efficient AI algorithm design, in-memory computing, adversarial, and trustworthy AI system.

Dr. Fan is the recipient of the National Science Foundation Career Award and several best paper awards. Dr. Fan is a senior member of ACM. Please refer to his research website for more details: <https://faculty.engineering.asu.edu/dfan/>



Chaitali Chakrabarti (Fellow, IEEE) received the M.S. and Ph.D. degrees in electrical engineering Department from the University of Maryland, College Park, MD, USA, in 1986 and 1990, respectively.

She is currently a Professor with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA. Her research interests are in the areas of low power embedded systems, algorithm-architecture co-design of signal processing systems, heterogeneous SoC systems, AI hardware, and federated learning.

Prof. Chakrabarti is the recipient of Distinguished Alumni Awards from University of Maryland in 2013 and IIT Kharagpur in 2018, multiple teaching awards at ASU, the Fulton Exemplar Award from 2014 to 2016, and the Palais Award in 2021.